

- [CPSC 375: High-Performance Computing](#)

[CPSC 375: High-Performance Computing](#)

Spring 2026

- [Syllabus](#)
- [Schedule](#)
- [Upload](#)
- [The Cluster Project](#)

Assignment 8

Due 11:59 p.m., Wednesday, April 22, 2026

IMPORTANT! This is an individual assignment. You may discuss broad issues of interpretation, understanding, and general approaches to a solution. However, the development of a specific solution or program code must be your own work. The assignment is expected to be entirely your own, designed and coded by you alone. If you need assistance, please consult your instructor or the TAs. Be sure to read the specific policies outlined in the [Academic Honesty in Computing](#) section.

Overview

The goal of this assignment is to implement a high-performance, stable radix sort for 32-bit unsigned integers. You will transition from a serial implementation to a distributed-memory environment using MPI. This assignment emphasizes stability, memory management, and identifying bottlenecks in parallel communication.

Definitions

To ensure clarity across distributed nodes, we define the following variables:

- N : The total number of integers in the entire dataset across all nodes (global count).
- n : The number of integers currently residing in the memory of a single MPI process (local count).
- b : The base of the sort. You are required to use $b=256$ (8 bits per digit).
- d : The number of sorting passes. For 32-bit integers and $b=256$, $d=4$.
- p : The number of MPI processes.

Part I: Serial Radix Sort

Before scaling your implementation to the cluster, you must first develop a robust serial Radix Sort.

Counting Sort. Write first a stable counting sort defined as

```
void counting_sort(uint32_t* input, uint32_t* output, int n, int pass);
```

Within this function, the loop implementation must iterate exactly n times, representing the total size of the array being processed. To enforce stability, you must iterate through the input array from right-to-left, specifically from index $n-1$ down to 0, when placing elements into the output buffer. For digit extraction, you are strictly prohibited from using the modulo or division operators; instead, you must utilize the bitwise shift $\gg$ and the bitwise-and-mask $\&0xFF$.

Radix Sort. Radix sort algorithm utilizes a divide-and-conquer strategy by sorting 32-bit unsigned integers one 8-bit byte at a time rather than attempting to sort the entire value in a single operation. The process involves an iterative approach where you call your `counting_sort()` function exactly four times, beginning with the least significant digit and progressing sequentially toward the most significant digit.

To optimize CPU performance and memory throughput, you are required to implement a double-buffering strategy. In this model, the output buffer from one pass is treated as the input for the subsequent pass, which effectively eliminates the need for expensive and redundant data copies between arrays. Because each underlying counting sort pass is strictly stable, the final cumulative result is a perfectly sorted list of 32-bit integers.

Part II: Parallel Radix Sort

To move from serial processing to a fully parallel setting, follow this sequence of actions:

1. Distributed fragments of input data so that each process initially holds only a local fragment of the n integers. No single process knows where its specific values belong within the global memory space of the cluster.
2. Construct local histograms (counters) on each node to identify how many of each digit prefix your local fragment contains.
3. Utilizing the cache, processes must communicate their local histograms to build a global map. Execute `MPI_Exscan` to calculate prefix sums across processes, allowing a process to determine how many instances of a digit exist on all preceding processes to find its own global starting offset.
4. Manually zero out the offsets on Rank 0 to correct the undefined result of an exclusive scan.
5. Run `MPI_Allreduce` simultaneously to compute the total global count for each digit, which establishes the boundaries across the processes.
6. Initiate `MPI_Alltoallv` to redistribute the numbers so that small numbers migrate toward Rank 0 and large numbers move toward the last rank.
7. Update your local count to reflect the new number of elements received during the shuffle.
8. Perform a local stable sort on this newly arrived data to fix the internal alignment, as the arrival order from various processors is not yet sorted.
9. Repeat this entire sequence for each of the four byte passes to ensure the data remains in the correct relative order as you move from the least significant bits to the final sorted sequence.

Part III: Benchmarking and Performance Analysis

To evaluate the efficiency of your parallel implementation on the Pine cluster, conduct a series of benchmarks designed to measure how the algorithm scales as you increase the computational resources.

Scaling Test. Perform a strong scaling analysis using a fixed dataset size of $N = 2^{28}$ integers (approximately 268 million elements). This size is large enough to exceed the 60MB L3 cache and saturate the 40Gbps InfiniBand interconnect, but small enough to run quickly without risking a system-wide memory swap. Record the execution time for the following process configurations:

Tests	Nodes	Tasks Per Node	Total Ranks	Environment Profile
I	1	1	1	Serial baseline (Single core)
II	1	12	12	Single socket (Shared L3 cache)
III	1	24	24	Dual socket (Intra-node / QPI)

IV	2	24	48	Distributed (InfiniBand / 2 nodes)
V	4	24	96	Standard test (4 nodes)

For each configuration, calculate and include the performance metrics (e.g., speedup, efficiency, etc) in your report.

Programming Notes. To ensure accurate benchmarking results:

- Use `MPI_Wtime()` to wrap only the sorting logic, excluding the time spent generating random numbers or performing I/O.
- Use `MPI_Barrier(MPI_COMM_WORLD)` immediately before starting the timer to ensure all ranks begin the sort at the same moment.
- Perform at least three runs for each configuration and report the average time to account for transient network spikes or cluster noise.
- Be aware that each rank handles approximately 11MB of raw data (2.8 million integers). Ensure your `malloc` calls accurately reflect the dynamic local count n to avoid exceeding the 256GB node limit.

Handin

Submit your source code and a brief report in PDF format, including your benchmarking table and performance analysis.

- **Welcome: Sean**

- [LogOut](#)

